claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-a7b50904be93527a2.jsonl`
- SHA-256: `aa0568ac6f1362151ac6805ba44e055db0b7458b24e31d41a3a38d745ba5d0cc`
- Source modified: `2026-07-02T08:17:19+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_793a5d86-1c4`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`

Transcript

1. user (2026-07-02T08:16:24.736Z)

You are reviewing OPEN GitHub PR #416 on Khelsutra/badminton-highlight-indexer to decide its fate: should we TAKE IT OVER (adopt/rebase/verify/merge) or CLOSE it as superseded by already-merged work?

PR #416: "fix(storage): gdrive-api chunked streaming download + safe idempotent put (R1-18)" (head branch: origin/fix/gdrive-api-chunked-download-safe-put, mergeable=MERGEABLE)
Investigation hint: MERGEABLE CLEAN. Streams via MediaIoBaseDownload in 32MiB chunks to .part then os.replace; fixes delete-before-upload data-loss window in put(). CHECK: does master backend/storage/gdrive_api.py still whole-file buffer (get_media().execute()) and still delete-before-upload? If master is unchanged there -> this is real still-needed work (take over clean).

Already MERGED onto master (verify against actual master code, do not assume):
- #422 = fix for --output-dir clobbering config.json output.output_dir (CODE_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC_STATUS/README/NEXT_STEPS registered; audit marked snapshot
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source_cache, reuses across Range requests, warms annotation proxy
- #426 = P4: bounded ffprobe/ffmpeg subprocess timeouts on serving paths
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mypy typing fix
master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (currently on master @ d00b4db). This is a SHARED checkout used by sibling sessions ? DO NOT checkout branches, DO NOT modify the working tree, DO NOT run git merge/rebase/add/commit. Read-only investigation only.

Do this:
1. Get the PR's intended change: run gh pr diff 416 -R Khelsutra/badminton-highlight-indexer (read the full diff). Also gh pr view 416 -R Khelsutra/badminton-highlight-indexer --json body,files,statusCheckRollup if useful.
2. Read the CURRENT master version of every file the PR touches (Read / Grep the files under C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer). To see the PR's branch version of a file without checkout, use: git -C "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer" show origin/fix/gdrive-api-chunked-download-safe-put:<path>.
3. Determine whether master ALREADY implements the PR's intent (superseded) or still lacks it. Cite concrete master file:line evidence either way. Do not trust the hint or handoff ? verify in code.
4. If CONFLICTING, explain precisely why (which files/hunks) and whether the conflict is because master already did the same thing (=> superseded/close) or because of adjacent edits (=> rebasable).
5. Judge correctness/quality of the PR change itself.

```
6. Recommend a fate:
   - CLOSE_SUPERSEDED: master already implements the intent; the PR adds nothing material.
   - TAKE_OVER_CLEAN: mergeable & still-needed; adopt and verify as-is.
   - TAKE_OVER_REBASE: still-needed but conflicting; needs a rebase onto master + re-verify,
then merge.
   - OWNER_DECISION: product/strategy/ambiguous ? needs the owner.

Return ONLY the structured object. Be precise and cite master file:line in master_evidence.
```

2. assistant (2026-07-02T08:16:27.540Z)

```
I'll investigate PR #416 to determine its fate. Let me start by getting the PR diff and
examining the current master code.
```

3. user (2026-07-02T08:16:30.390Z)

```
diff --git a/backend/storage/gdrive_api.py b/backend/storage/gdrive_api.py
index 210edce..3252e39 100644
--- a/backend/storage/gdrive_api.py
+++ b/backend/storage/gdrive_api.py
@@ -18,9 +18,12 @@

 **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-native Docs
 (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of scope;
export
-them first). ``put`` is idempotent (deletes an existing file at the same path before creating).
Path
-resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files under one
folder
-(a rare pre-existing state this backend never creates), resolution returns the first match.
+them first). ``put`` is idempotent: an existing file at the same path is overwritten **in
place** via
+``files().update`` (same file id; the old bytes survive until the new upload succeeds ? never
+delete-then-create). Path resolution keys on ``(name, parent)``; if Drive already holds
+duplicate-named files under one folder (a rare pre-existing state this backend never creates),
+resolution returns the first match. Downloads stream in chunks (``MediaIoBaseDownload``) ? a
+multi-GB reel is never buffered whole in RAM.
 """

 from __future__ import annotations
@@ -35,6 +38,9 @@

 _FOLDER_MIME = "application/vnd.google-apps.folder"
 _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
+# Download chunk size: bounds fetch_to_local RAM no matter the object size (vs. the
googleapiclient
+# default of 100 MiB) while keeping the number of round-trips per multi-GB reel reasonable.
+_DOWNLOAD_CHUNK_BYTES = 32 * 1024 * 1024


 def _q_escape(name: str) -> str:
@@ -83,6 +89,18 @@ def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
             local_path, mimetype=content_type, resumable=True
         )  # pragma: no cover

+    def _downloader(self, fh: Any, request: Any) -> Any:
+        """The chunked-download driver for a ``files().get_media`` request ? lazy
+        ``MediaIoBaseDownload`` (real path). A test seam: monkeypatch this to avoid the
+        ``googleapiclient`` dependency (the fakes drive the same ``next_chunk()`` loop)."""
+        from googleapiclient.http import (
+            MediaIoBaseDownload,  # type: ignore  # pragma: no cover
+        )
+
+        return MediaIoBaseDownload(
+            fh, request, chunksize=_DOWNLOAD_CHUNK_BYTES
```

```
+        )  # pragma: no cover
+
     # -- path -> id resolution
---------------------------------------------------------------
     def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
         """Walk ``key``'s path components from the root folder, returning the leaf's file id
(or
@@ -140,12 +158,24 @@ def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
         file_id = self._resolve_id(ref.key)
         if not file_id:
             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
-        data = (
-            self._service.files().get_media(fileId=file_id).execute()
-        )  # alt=media ? raw bytes
+        request = self._service.files().get_media(fileId=file_id)  # alt=media ? bytes
         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
-        with io.open(dst, "wb") as fh:
-            fh.write(data)
+        # Stream in bounded chunks to a ``.part`` sibling, then rename into place: a multi-GB
+        # reel never sits whole in RAM (the old ``get_media().execute()`` buffered the entire
+        # object as one bytes value ? OOM fodder on a memory-limited Cloud Run box), and a
+        # mid-download failure can't leave a truncated ``dst`` that a later ``overwrite=False``
+        # call would trust as complete.
+        tmp = dst + ".part"
+        try:
+            with io.open(tmp, "wb") as fh:
+                downloader = self._downloader(fh, request)
+                done = False
+                while not done:
+                    _, done = downloader.next_chunk()
+            os.replace(tmp, dst)
+        finally:
+            if os.path.exists(tmp):
+                os.remove(tmp)
         return dst

     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
@@ -158,16 +188,22 @@ def put(self, local_path, ref, *, content_type=None) -> StorageRef:
         if parent_id is None:
             raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
         # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a re-put
would
-        # ORPHAN the prior file (and reads would keep returning the stale first match). Delete
an
-        # existing file at this exact path first, so put() overwrites rather than accumulates.
+        # ORPHAN the prior file (and reads would keep returning the stale first match).
Overwrite
+        # an existing file at this exact path IN PLACE via files().update ? the file id is kept
+        # and the OLD bytes survive until the NEW upload succeeds. (The previous delete-then-
create
+        # ordering lost the object outright when the create failed mid-upload; create-first-
then-
+        # delete would instead open a duplicate-(name, parent) window where reads could resolve
+        # either copy.)
+        media = self._media_body(str(local_path), content_type)
         existing = self._resolve_id(ref.key)
         if existing:
-            self._service.files().delete(fileId=existing).execute()
-        self._service.files().create(
-            body={"name": name, "parents": [parent_id]},
-            media_body=self._media_body(str(local_path), content_type),
-            fields="id",
-        ).execute()
+            self._service.files().update(fileId=existing, media_body=media).execute()
+        else:
```

```
+            self._service.files().create(
+                body={"name": name, "parents": [parent_id]},
+                media_body=media,
+                fields="id",
+            ).execute()
         return ref

     def exists(self, ref) -> bool:
diff --git a/tests/test_storage_gdrive_api.py b/tests/test_storage_gdrive_api.py
index c89af22..44b001f 100644
--- a/tests/test_storage_gdrive_api.py
+++ b/tests/test_storage_gdrive_api.py
@@ -2,8 +2,10 @@

 Fully offline: a tiny in-memory ``_FakeDrive`` stands in for the googleapiclient Drive v3
``service``
 (injected, mirroring the GCS fake-client tests). No ``google-api-python-client`` / network
needed.
-Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, and the full
-path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete).
+Asserts: scheme parsing, registry wiring, the hyphen?underscore settings lookup, the full
+path-keyed round-trip (put ? exists ? stat ? fetch ? list ? delete), the chunked (streaming)
+download path, and overwrite safety: a re-put updates in place (same file id) and a mid-upload
+failure leaves the OLD object intact ? never delete-before-upload.
 """

 import re
@@ -26,6 +28,17 @@ def execute(self):
         return self._value


+class _RaiseExec:
+    """An operation whose ``execute()`` fails (simulated network/quota mid-upload error) ?
+    crucially WITHOUT having mutated the fake Drive first."""
+
+    def __init__(self, msg):
+        self._msg = msg
+
+    def execute(self):
+        raise RuntimeError(self._msg)
+
+
 class _FakeFiles:
     def __init__(self, drive):
         self.d = drive
@@ -46,6 +59,8 @@ def list(self, q="", spaces=None, fields=None, pageToken=None):
         return _Exec({"files": out})  # one page, no nextPageToken

     def create(self, body=None, media_body=None, fields=None):
+        if media_body is not None and self.d.fail_uploads:
+            return _RaiseExec("simulated mid-upload failure (network/quota)")
         nid = self.d.new_id()
         self.d.nodes[nid] = {
             "name": body["name"],
@@ -59,6 +74,13 @@ def create(self, body=None, media_body=None, fields=None):
                 self.d.content[nid] = fh.read()
         return _Exec({"id": nid})

+    def update(self, fileId=None, media_body=None, fields=None):
+        if self.d.fail_uploads:
+            return _RaiseExec("simulated mid-upload failure (network/quota)")
+        with open(media_body, "rb") as fh:  # _media_body patched ? the local path
+            self.d.content[fileId] = fh.read()
+        return _Exec({"id": fileId})
+
```

```
     def get_media(self, fileId=None):
         return _Exec(self.d.content.get(fileId, b""))

@@ -85,6 +107,9 @@ def __init__(self):
         self.nodes = {"root": {"name": "root", "mimeType": "folder", "parents": []}}
         self.content = {}
         self._n = 0
+        self.fail_uploads = (
+            False  # flip on to make the NEXT upload (create/update) fail
+        )

     def new_id(self):
         self._n += 1
@@ -94,11 +119,36 @@ def files(self):
         return _FakeFiles(self)


+class _FakeDownloader:
+    """Stands in for ``googleapiclient.http.MediaIoBaseDownload``: drains the fake
``get_media``
+    request's bytes into the file handle a few bytes per ``next_chunk()`` call, so the
+    production streaming loop is exercised chunk by chunk (never one whole-file buffer)."""
+
+    chunk_size = 4
+
+    def __init__(self, fh, request):
+        self._fh = fh
+        self._data = request.execute()  # the fake request; a real one streams over HTTP
+        self._pos = 0
+        self.chunks = 0
+
+    def next_chunk(self):
+        piece = self._data[self._pos : self._pos + self.chunk_size]
+        self._fh.write(piece)
+        self._pos += len(piece)
+        self.chunks += 1
+        return None, self._pos >= len(self._data)
+
+
 @pytest.fixture
 def be(monkeypatch):
-    """A GDriveApiStorage on a fake service, with _media_body patched to the local path (no
-    googleapiclient dependency ? the fake `create` reads the bytes from that path)."""
+    """A GDriveApiStorage on a fake service, with the two googleapiclient seams patched:
+    _media_body ? the local path (the fake `create`/`update` read the bytes from it) and
+    _downloader ? _FakeDownloader (drives the real next_chunk() streaming loop)."""
     monkeypatch.setattr(GDriveApiStorage, "_media_body", lambda self, p, ct: p)
+    monkeypatch.setattr(
+        GDriveApiStorage, "_downloader", lambda self, fh, req: _FakeDownloader(fh, req)
+    )
     return GDriveApiStorage(service=_FakeDrive())


@@ -163,7 +213,7 @@ def test_roundtrip_put_exists_stat_fetch_list_delete(be, tmp_path):


 def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
-    """Re-putting the same path OVERWRITES (delete-then-create) ? Drive allows duplicate names,
so a
+    """Re-putting the same path OVERWRITES (update-in-place) ? Drive allows duplicate names, so
a
     naive create would orphan the old file and reads would keep returning the stale one."""
     ref = parse_uri("gdrive-api://Coaching/clip.mp4")
     v1 = tmp_path / "v1.mp4"
@@ -181,6 +231,98 @@ def test_put_is_idempotent_no_duplicate_on_reput(be, tmp_path):
```

```
    )  # the latest bytes, not the orphaned first


+def test_reput_updates_in_place_preserving_file_id(be, tmp_path):
+    """The idempotent overwrite is files().update on the EXISTING id ? the Drive file id is
+    stable across re-puts (shared links / ids held elsewhere keep working)."""
+    ref = parse_uri("gdrive-api://Coaching/clip.mp4")
+    v1 = tmp_path / "v1.mp4"
+    v1.write_bytes(b"FIRST")
+    v2 = tmp_path / "v2.mp4"
+    v2.write_bytes(b"SECOND")
+    be.put(str(v1), ref)
+    id_before = be._resolve_id(ref.key)
+    be.put(str(v2), ref)
+    assert be._resolve_id(ref.key) == id_before
+
+
+def test_put_failure_mid_upload_keeps_old_object_intact(be, tmp_path):
+    """A re-put whose upload fails must leave the PREVIOUS object readable. The old
+    delete-then-create ordering deleted first, so a failed create lost the reel at both ends;
+    update-in-place only replaces content once the upload succeeds."""
+    ref = parse_uri("gdrive-api://Coaching/reel.mp4")
+    v1 = tmp_path / "v1.mp4"
+    v1.write_bytes(b"OLD-REEL")
+    v2 = tmp_path / "v2.mp4"
+    v2.write_bytes(b"NEW-REEL")
+    be.put(str(v1), ref)
+
+    be._service.fail_uploads = True
+    with pytest.raises(RuntimeError, match="mid-upload"):
+        be.put(str(v2), ref)
+
+    assert be.exists(ref)  # still there ?
+    dst = tmp_path / "out.mp4"
+    be.fetch_to_local(ref, str(dst))
+    assert dst.read_bytes() == b"OLD-REEL"  # ? with the old bytes intact
+
+    be._service.fail_uploads = False  # and a retry succeeds: still exactly one file
+    be.put(str(v2), ref)
+    listed = [r.uri for r in be.list(parse_uri("gdrive-api://Coaching"))]
+    assert listed == ["gdrive-api://Coaching/reel.mp4"]
+    dst2 = tmp_path / "out2.mp4"
+    be.fetch_to_local(ref, str(dst2))
+    assert dst2.read_bytes() == b"NEW-REEL"
+
+
+def test_fetch_large_file_streams_in_chunks(be, tmp_path, monkeypatch):
+    """A file much larger than the chunk size is drained via MANY next_chunk() calls ? the
+    production loop streams; it never materializes the object as one .execute() buffer."""
+    data = b"0123456789" * 5  # 50 bytes ? the fake 4-byte chunk
+    src = tmp_path / "big.mp4"
+    src.write_bytes(data)
+    ref = parse_uri("gdrive-api://Coaching/big.mp4")
+    be.put(str(src), ref)
+
+    made = []
+
+    def spy(self, fh, req):
+        d = _FakeDownloader(fh, req)
+        made.append(d)
+        return d
+
+    monkeypatch.setattr(GDriveApiStorage, "_downloader", spy)
+    dst = tmp_path / "out" / "big.mp4"
+    be.fetch_to_local(ref, str(dst))
```

```
+    assert dst.read_bytes() == data  # byte-identical after reassembly
+    assert made[0].chunks == 13  # ceil(50 / 4) ? chunked, not one whole-file buffer
+    assert not (tmp_path / "out" / "big.mp4.part").exists()  # temp renamed away
+
+
+def test_fetch_failure_mid_download_leaves_no_truncated_dst(be, tmp_path, monkeypatch):
+    """If the stream dies mid-download, dst must NOT exist (a truncated file would be trusted
+    as complete by a later overwrite=False call) and the .part temp is cleaned up."""
+
+    class _Boom(_FakeDownloader):
+        def next_chunk(self):
+            if self.chunks >= 2:
+                raise RuntimeError("simulated mid-download network failure")
+            return super().next_chunk()
+
+    src = tmp_path / "big.mp4"
+    src.write_bytes(b"0123456789ABCDEF")
+    ref = parse_uri("gdrive-api://Coaching/big.mp4")
+    be.put(str(src), ref)
+
+    monkeypatch.setattr(
+        GDriveApiStorage, "_downloader", lambda self, fh, req: _Boom(fh, req)
+    )
+    dst = tmp_path / "out.mp4"
+    with pytest.raises(RuntimeError, match="mid-download"):
+        be.fetch_to_local(ref, str(dst))
+    assert not dst.exists()
+    assert not (tmp_path / "out.mp4.part").exists()
+
+
 def test_fetch_missing_raises(be, tmp_path):
     with pytest.raises(FileNotFoundError):
         be.fetch_to_local(
```

4. user (2026-07-02T08:16:31.995Z)


```
{"body":"## What / why\n\nFixes audit finding **R1-18** (AUDIT_REPORT_khelsutra-guru_2026-07-02)
on `backend/storage/gdrive_api.py` ? two latent hazards on the Drive-API storage hot path before
the reel round-trip goes live:\n\n1. **Whole-file RAM buffering on download.** `fetch_to_local`
did `get_media().execute()`, materializing the entire object as one `bytes` value ? an OOM
waiting to happen for a multi-GB source/reel on a memory-limited Cloud Run box. It now streams
via `googleapiclient.http.MediaIoBaseDownload` in bounded **32 MiB chunks** (lazy import behind
a `_downloader` seam, mirroring the existing `_media_body` seam/convention). The stream writes
to a `dst + \".part\"` sibling and `os.replace`s into place, so a mid-download failure can never
leave a truncated `dst` that a later `overwrite=False` call would trust as complete.\n\n2.
**Delete-before-upload data-loss window in `put()`.** A re-put deleted the existing file at the
path and only then ran `files().create(...)` ? if the resumable create failed (network/quota),
the reel was gone at both ends. Per the audit verifier's refinement, re-puts now overwrite **in
place** via `files().update(fileId=existing, media_body=...)`: the old bytes survive until the
new upload succeeds, the Drive file id is preserved across re-puts, and no duplicate-`(name,
parent)` window opens (which a create-first-then-delete ordering would, given path resolution
returns the first match among duplicates). Fresh paths still `files().create(...)`. The
**idempotent-put contract from PR #338** (exactly one file per path, latest bytes win) is
preserved and still covered by the pre-existing test.\n\nNo config knobs, no behavior change for
callers: same `StorageBackend` interface, same idempotence semantics ? only the failure modes
improve.\n\n## Verification\n\nRan locally in an isolated worktree on top of `origin/master`
(e410136):\n\n- `python -m pytest tests/test_storage_gdrive_api.py -q` ? **14 passed** (was 10;
4 new tests: `test_fetch_large_file_streams_in_chunks` (50-byte file / 4-byte fake chunks ? 13
`next_chunk()` calls, byte-identical reassembly),
`test_fetch_failure_mid_download_leaves_no_truncated_dst`,
`test_reput_updates_in_place_preserving_file_id`,
`test_put_failure_mid_upload_keeps_old_object_intact` (old bytes readable after a simulated mid-
upload failure; retry succeeds with exactly one file)).\n- `python -m pytest tests -q -k
storage` ? **57 passed, 2 skipped** (the full storage-backend surface).\n- `ruff check` + `ruff
format --check` clean on both changed files; `mypy backend/storage/gdrive_api.py` clean.\n\nNot
```

verifiable locally: the real `googleapiclient` path (`MediaIoBaseDownload` / `MediaFileUpload` / real Drive) ? the library is deliberately absent from the test env; the fakes drive the identical `next_chunk()` loop and `files().update`/`create` call shapes. Full suite + coverage floor is CI's job (`run_all_tests.py` lane).\n\n? Generated with [Claude Code](https://claude.com/claude-code)","files":[{"path":"backend/storage/gdrive_api.py","additions":52,"deletions":16,"changeType":"MODIFIED"},{"path":"tests/test_storage_gdrive_api.py","additions":147,"deletions":5,"changeType":"MODIFIED"}],"headRefName":"fix/gdrive-api-chunked-download-safe-put","mergeable":"MERGEABLE","statusCheckRollup":[{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:36Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324758","name":"Ruff lint","startedAt":"2026-07-02T04:53:25Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:33Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324711","name":"Leak scan (PII / attribution guard)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:29Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324738","name":"Markdown link check (live docs)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:57Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324720","name":"Mypy (lenient baseline)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:46Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324746","name":"Syntax sweep + import smoke (no GPU stack)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:56:19Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324722","name":"Full test suite (offline, mocked)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:50Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324721","name":"Golden fixtures cross-OS (ubuntu-latest)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:54:35Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324736","name":"Golden fixtures cross-OS (windows-latest)","startedAt":"2026-07-02T04:53:25Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:51Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324741","name":"Golden fixtures cross-OS (macos-latest)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:53:58Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324780","name":"Dependency license scan (no copyleft in runtime deps)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"},{"__typename":"CheckRun","completedAt":"2026-07-02T04:54:25Z","conclusion":"SUCCESS","detailsUrl":"https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566343537/job/84694324734","name":"Built wheel smoke (install + launch)","startedAt":"2026-07-02T04:53:24Z","status":"COMPLETED","workflowName":"CI"}]}

5. assistant (2026-07-02T08:16:36.241Z)

Now let me read the current master version of the file the PR touches.

6. user (2026-07-02T08:16:38.948Z)

```
d00b4db fix gcs storage lazy import typing (#428)
---
b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
```

7. user (2026-07-02T08:16:38.965Z)

```
1       """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api://`` (M12, D14
Northstar).
2
3       The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
4       Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive://``
backend, which
5       is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6       Cloud Run / GPU box uses.
7
8       ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9       ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10      ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11      **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12      fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14      ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15      (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16      consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
? this backend
17      ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18
19      **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
native Docs
20      (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
scope; export
21      them first). ``put`` is idempotent (deletes an existing file at the same path before
creating). Path
22      resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files
under one folder
23      (a rare pre-existing state this backend never creates), resolution returns the first
match.
24      """
25
26      from __future__ import annotations
27
28      import io
29      import os
30      from datetime import datetime
31      from typing import Any, Iterator, Optional
32
33      from backend.storage.base import StorageBackend
34      from backend.storage.ref import StorageRef, StorageStat
35
36      _FOLDER_MIME = "application/vnd.google-apps.folder"
```

```
37      _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
38
39
40      def _q_escape(name: str) -> str:
41          """Escape a value for a Drive ``q`` string literal (single-quoted): ``\\`` then
``'``."""
42          return name.replace("\\", "\\\\").replace("'", "\\'")
43
44
45      class GDriveApiStorage(StorageBackend):
46          """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one
47          from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths."""
48
49          def __init__(
50              self, *, service: Any = None, root_folder_id: str = "root", **_ignored
51          ) -> None:
52              self._root_id = root_folder_id or "root"
53              if service is not None:
54                  self._service = service
55                  return
56              self._service = self._build_service()
57
58          # -- construction (real path; lazy Google imports)
---------------------------------------
59          def _build_service(self) -> Any:
60              try:
61                  import google.auth  # type: ignore
62                  from googleapiclient.discovery import build  # type: ignore
63              except (
64                  ImportError
65              ) as e:  # pragma: no cover - exercised only without the extra installed
66                  raise RuntimeError(
67                      "the gdrive-api backend needs the 'storage-gdrive-api' extra "
68                      "(google-api-python-client + google-auth). Install it on a host that
uses "
69                      "gdrive-api:// URIs; credentials come from ADC (a service account /
workload "
70                      "identity over the Drive scope) ? never from config."
71                  ) from e
72              creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
73              return build("drive", "v3", credentials=creds, cache_discovery=False)
74
75          def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
76              """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
(real path).
77              A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
78              from googleapiclient.http import (
79                  MediaFileUpload,  # type: ignore  # pragma: no cover
80              )
81
82              return MediaFileUpload(
83                  local_path, mimetype=content_type, resumable=True
84              )  # pragma: no cover
85
86          # -- path -> id resolution
------------------------------------------------------------
87          def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
88              """Walk ``key``'s path components from the root folder, returning the leaf's
file id (or
89              ``None`` if a component is missing). With ``create_dirs`` EVERY missing
component is created as
90              a folder ? ``put`` uses this on the parent-directory path (where all components
are folders),
```

```
            so a write into a fresh subfolder works. The read paths leave it ``False`` (a
missing component
            ? ``None``), never creating anything."""
            parent = self._root_id
            parts = [p for p in key.split("/") if p]
            for part in parts:
                q = (
                    f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
                    f"and trashed = false"
                )
                found = (
                    self._service.files()
                    .list(q=q, spaces="drive", fields="files(id, mimeType)")
                    .execute()
                    .get("files", [])
                )
                if found:
                    parent = found[0]["id"]
                elif create_dirs:
                    parent = (
                        self._service.files()
                        .create(
                            body={
                                "name": part,
                                "mimeType": _FOLDER_MIME,
                                "parents": [parent],
                            },
                            fields="id",
                        )
                        .execute()["id"]
                    )
                else:
                    return None
            return parent

        def _split(self, key: str) -> tuple[str, str]:
            """Split a ``gdrive-api://`` key into (parent dir path, leaf name)."""
            clean = key.strip("/")
            head, _, name = clean.rpartition("/")
            return head, name

        # -- StorageBackend interface
------------------------------------------------------------
        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError(
                    "GDriveApiStorage.fetch_to_local requires a dst path (no local mount)"
                )
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            data = (
                self._service.files().get_media(fileId=file_id).execute()
            )  # alt=media ? raw bytes
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            with io.open(dst, "wb") as fh:
                fh.write(data)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            parent_path, name = self._split(ref.key)
            parent_id = (
```

```
                self._resolve_id(parent_path, create_dirs=True)
                if parent_path
                else self._root_id
            )
            if parent_id is None:
                raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
            # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a
re-put would
            # ORPHAN the prior file (and reads would keep returning the stale first match).
Delete an
            # existing file at this exact path first, so put() overwrites rather than
accumulates.
            existing = self._resolve_id(ref.key)
            if existing:
                self._service.files().delete(fileId=existing).execute()
            self._service.files().create(
                body={"name": name, "parents": [parent_id]},
                media_body=self._media_body(str(local_path), content_type),
                fields="id",
            ).execute()
            return ref

        def exists(self, ref) -> bool:
            return self._resolve_id(ref.key) is not None

        def stat(self, ref) -> StorageStat:
            file_id = self._resolve_id(ref.key)
            if not file_id:
                raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
            meta = (
                self._service.files()
                .get(fileId=file_id, fields="size, modifiedTime, md5Checksum, mimeType")
                .execute()
            )
            modified = None
            if meta.get("modifiedTime"):
                # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
                modified = datetime.fromisoformat(
                    meta["modifiedTime"].replace("Z", "+00:00")
                )
            return StorageStat(
                size=int(meta.get("size") or 0),
                modified=modified,
                etag=meta.get("md5Checksum"),
                content_type=meta.get("mimeType"),
            )

        def list(self, prefix) -> Iterator[StorageRef]:
            folder_id = (
                self._resolve_id(prefix.key) if prefix.key.strip("/") else self._root_id
            )
            if not folder_id:
                return
            base = prefix.key.strip("/")
            page_token: Optional[str] = None
            while True:
                resp = (
                    self._service.files()
                    .list(
                        q=f"'{_q_escape(folder_id)}' in parents and trashed = false",
                        spaces="drive",
                        fields="nextPageToken, files(id, name)",
                        pageToken=page_token,
                    )
```

```
215                     .execute()
216                 )
217                 for f in resp.get("files", []):
218                     key = f"{base}/{f['name']}" if base else f["name"]
219                     yield StorageRef("gdrive-api", "", key, f"gdrive-api://{key}")
220                 page_token = resp.get("nextPageToken")
221                 if not page_token:
222                     break
223 
224         def delete(self, ref) -> None:
225             file_id = self._resolve_id(ref.key)
226             if file_id:
227                 self._service.files().delete(fileId=file_id).execute()
228 
```

8. assistant (2026-07-02T08:16:45.610Z)

The master version is definitive. Let me confirm the key facts:

- Master `fetch_to_local` (lines 143-148): still does `get_media(fileId=file_id).execute()` ? whole-file buffer into `bytes`, then writes directly to `dst` (no `.part` temp, no chunked streaming).
- Master `put` (lines 160-171): still uses **delete-before-create** ? `files().delete(fileId=existing).execute()` then `files().create(...)`. The delete-before-upload data-loss window is present.
- No `_downloader` seam and no `_DOWNLOAD_CHUNK_BYTES` constant in master.

The file history confirms master's `gdrive_api.py` was last touched by #338 (feature intro), #360 (isort), and #387 (ruff format) ? none of the merged PRs #422-#428 touched this file's download/put logic. Let me verify the merged PRs didn't touch this file, and check the test file's current state.

9. user (2026-07-02T08:16:49.806Z)

```
=== grep get_media / delete-then-create in master ===
backend/storage/gdrive_api.py:94:        parts = [p for p in key.split("/") if p]
backend/storage/gdrive_api.py:95:        for part in parts:
backend/storage/gdrive_api.py:97:                f"name = '{_q_escape(part)}' and
'{_q_escape(parent)}' in parents "
backend/storage/gdrive_api.py:113:                                "name": part,
backend/storage/gdrive_api.py:128:        head, _, name = clean.rpartition("/")
backend/storage/gdrive_api.py:144:
self._service.files().get_media(fileId=file_id).execute()
backend/storage/gdrive_api.py:165:
self._service.files().delete(fileId=existing).execute()
backend/storage/gdrive_api.py:227:
self._service.files().delete(fileId=file_id).execute()
=== last commit touching test file ===
b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
=== does master test have new tests? ===
tests/test_storage_gdrive_api.py:108:def test_parse_uri_gdrive_api_is_path_keyed():
tests/test_storage_gdrive_api.py:118:def test_unknown_scheme_still_rejected():
tests/test_storage_gdrive_api.py:123:def test_config_accepts_gdrive_api_backend():
tests/test_storage_gdrive_api.py:131:def
test_get_storage_wires_gdrive_api_and_reads_underscore_settings(monkeypatch):
tests/test_storage_gdrive_api.py:144:def test_roundtrip_put_exists_stat_fetch_list_delete(be,
tmp_path):
tests/test_storage_gdrive_api.py:165:def test_put_is_idempotent_no_duplicate_on_reput(be,
tmp_path):
tests/test_storage_gdrive_api.py:184:def test_fetch_missing_raises(be, tmp_path):
tests/test_storage_gdrive_api.py:191:def test_fetch_requires_dst(be):
tests/test_storage_gdrive_api.py:196:def test_put_at_root_uses_root_folder(be, tmp_path):
tests/test_storage_gdrive_api.py:204:def
```

test_build_service_without_libs_raises_actionable_error(monkeypatch):

10. assistant (2026-07-02T08:17:01.504Z)

Confirmed. Master state is definitive:

- **`fetch_to_local`**: master line 144 still `self._service.files().get_media(fileId=file_id).execute()` ? whole-file buffer. No `MediaIoBaseDownload`, no `_downloader`, no `_DOWNLOAD_CHUNK_BYTES`, no `.part` temp anywhere in the file.
- **`put`**: master line 165 still `self._service.files().delete(fileId=existing).execute()` followed by `files().create(...)` at line 166 ? the delete-before-upload data-loss window. No `files().update` in master.
- **Tests**: master has 10 test functions; none of the 4 new tests from the PR (`test_reput_updates_in_place_preserving_file_id`, `test_put_failure_mid_upload_keeps_old_object_intact`, `test_fetch_large_file_streams_in_chunks`, `test_fetch_failure_mid_download_leaves_no_truncated_dst`) are present.

None of the merged PRs #422-#428 touched `gdrive_api.py` (last touched by #338/#360/#387, all pre-master-HEAD infra). So the PR's intent is entirely absent from master ? **not superseded**.

Now let me judge correctness/quality of the PR change itself. The `put()` update-in-place logic and the `fetch_to_local` chunked streaming with `.part` + `os.replace` are both sound. Let me double-check one subtlety in the `finally` block of `fetch_to_local` in the PR ? the ordering of `os.replace` vs `finally` cleanup.

The PR's `fetch_to_local`:
```python
tmp = dst + ".part"
try:
    with io.open(tmp, "wb") as fh:
        downloader = self._downloader(fh, request)
        done = False
        while not done:
            _, done = downloader.next_chunk()
    os.replace(tmp, dst)
finally:
    if os.path.exists(tmp):
        os.remove(tmp)
```

On success, `os.replace(tmp, dst)` moves `tmp` to `dst`, so `tmp` no longer exists and the `finally` cleanup is a no-op. On failure, `tmp` still exists and is removed, leaving no truncated file. This is correct. The `put()` change (compute `media` once, `update` if existing else `create`) is also correct and preserves the idempotent-put contract.

CI is fully green (all 12 checks SUCCESS including the full offline test suite), and mergeable is MERGEABLE. This is real, still-needed work that applies cleanly.

11. user (2026-07-02T08:17:19.051Z)

Structured output provided successfully